

Laboratório Prático — Pentest Web

Utilizando Apenas DevTools

Status	Método	Domínio	Arquivo	Iniciador
200	POST	o4507096022450176.ingest.d...	/api/4507096429756496/envelope/?sentry_version=7&sentry_key=175180b5f191796714d2f9138c06c...	fetch
200	POST	api.segment.io	p	12985.de00c2fb.js:22 (fetch)
200	POST	api.segment.io	i	12985.de00c2fb.js:22 (fetch)
200	GET	downloads.intercomcdn.com	9e0f012f15b6fc981dde2f1f5198d728.png	img
101	GET	nexus-websocket-a.intercom.io	5-aHmG3P0ulokU9ZYnpbAMe0BqOfPFxWvnEsG0Jj3g26PuuerAbPBjzDWoCyJCMds1S8VNfyi-L_U07h8N...	vendor.99fe2e83.js:2 (websocket)
200	POST	api.segment.io	m	12985.de00c2fb.js:22 (fetch)

Cenário

Fui contratado pela empresa fictícia BlueRiver Bank para realizar uma avaliação de segurança em sua nova plataforma bancária web antes do lançamento oficial.

O escopo autorizado incluía:

- página de login
- painel do cliente
- sistema de transferências
- upload de comprovantes
- APIs utilizadas pelo frontend

A empresa queria descobrir:

- quais informações um atacante poderia obter usando apenas o navegador
- se o frontend expunha dados sensíveis
- se existiam falhas exploráveis sem ferramentas avançadas

Meu objetivo era conduzir toda a análise utilizando apenas o:

- Chrome DevTools

Etapas 1 — Reconhecimento Inicial

Assim que acessei o site da BlueRiver Bank, minha primeira ação foi abrir o DevTools.

Utilizei:

F12 ou CTRL + SHIFT + I

O objetivo inicial era simples:

- entender como o sistema funcionava
- identificar tecnologias utilizadas
- observar o comportamento do frontend

Comecei analisando as abas:

- Elements
- Network
- Sources
- Application

Etapa 2 — Investigando o HTML

Na aba:

Elements

comecei a procurar:

- comentários esquecidos
- elementos ocultos
- referências internas

Durante a análise encontrei o seguinte comentário:

```
<!-- TODO: remover painel antigo antes da release -->
```

```
<!-- /legacy-admin -->
```

Aquilo chamou imediatamente minha atenção.

Meu pensamento foi:

“Se o comentário ainda existe, talvez a rota também exista.”

Então tentei acessar manualmente:

<https://blueriver-bank.com/legacy-admin>

Resultado:

- a página realmente existia
- o sistema exibiu um login administrativo antigo

O que isso demonstrava?

A aplicação possuía:

- funcionalidades legadas expostas
- rotas esquecidas
- possível superfície adicional de ataque

Um cibercriminoso poderia:

- tentar credenciais padrão
- procurar vulnerabilidades antigas
- explorar componentes sem manutenção

Etapas 3 — Analisando Elementos Ocultos

Ainda na aba Elements encontrei um botão escondido:

```
<button style="display:none">
```

Administração

```
</button>
```

Removi manualmente:

display:none

O botão apareceu na tela.

Ao clicar:

- fui redirecionado para /admin-panel

Porém o backend bloqueou o acesso.

Conclusão da análise

O frontend escondia funcionalidades apenas visualmente.

Isso mostrou que:

- ocultar elementos NÃO é segurança
- qualquer usuário pode alterar o HTML localmente

Etapa 4 — Investigando as Requisições da Aplicação

A próxima etapa foi abrir:

Network

e filtrar apenas:

Fetch/XHR

Meu objetivo era observar:

- chamadas da API
- parâmetros enviados
- dados retornados pelo servidor

Etapa 5 — Analisando o Processo de Login

Realizei login com uma conta de teste fornecida pela empresa.

Observei a seguinte requisição:

```
POST /api/login
```

Resposta recebida:

```
{  
  
  "token": "eyJhbGciOiJIUzI1Ni...",  
  
  "user": "carlos",  
  
  "role": "user"  
}
```

Aquilo já revelava:

- uso de JWT
- autenticação baseada em token
- estrutura da API

Etapa 6 — Descobrendo Endpoints Internos

Continuando a análise no Network, identifiquei outra chamada interessante:

```
GET /api/internal/users
```

Aquilo não deveria aparecer para um usuário comum.

Abri a resposta da requisição e encontrei:

```
[
{
  "name":"Carlos Lima",
  "email":"carlos@blueriver.com"
},
{
  "name":"Fernanda Rocha",
  "email":"fernanda@blueriver.com"
}
]
```

O que isso significava?

A aplicação estava expondo:

- emails corporativos
- nomes internos
- estrutura de usuários

Um atacante poderia utilizar essas informações para:

- phishing
- engenharia social
- ataques direcionados

Etapa 7 — Testando Controle de Acesso

Durante a navegação percebi uma requisição:

```
GET /api/account?id=1008
```

Minha hipótese:

“Talvez o backend não valide corretamente o proprietário da conta.”

Cliquei na requisição:

- botão direito
- Edit and Resend

Alterei:

id=1008

para:

id=1001

Resultado:

- o servidor retornou dados de outro cliente

Vulnerabilidade identificada

IDOR

(Insecure Direct Object Reference)

Dados acessados

A resposta continha:

- nome completo
- saldo bancário
- histórico de transações
- CPF parcial

Impacto

Um criminoso poderia:

- acessar contas de terceiros
- coletar dados financeiros

- realizar fraude
- violar LGPD

Etapa 8 — Investigando o JavaScript

Acessei a aba:

Sources

e comecei a analisar os arquivos JavaScript carregados.

Encontrei:

- main.js
- admin.js
- config.js

Dentro do arquivo `config.js` encontrei:

```
const API_URL = "https://dev-api.blueriver.internal";
```

O que isso revelava?

O frontend estava expondo:

- ambiente interno
- infraestrutura da empresa
- possíveis APIs de desenvolvimento

Outra descoberta importante

Mais abaixo encontrei:

```
const FIREBASE_KEY = "AlzaSyXXXXXX";
```

Risco identificado

Uma chave exposta poderia permitir:

- abuso da API
- consumo indevido
- coleta de informações

Etapa 9 — Investigando Armazenamento Local

Acessei:

Application

e analisei:

- Local Storage
- Session Storage
- Cookies

No Local Storage encontrei:

```
token = eyJhbGcOiJIUzI1Ni...
```

Problema identificado

O token JWT estava acessível ao JavaScript.

Isso significa que:

- um XSS poderia roubar sessões
- contas poderiam ser sequestradas

Etapa 10 — Testando Possível XSS

Encontrei um campo de pesquisa na aplicação.

Digitei:

```
<script>alert('XSS')</script>
```

Resultado:

- o alerta executou no navegador

Conclusão

A aplicação era vulnerável a:

XSS Refletido

O que um criminoso poderia fazer?

- roubar cookies
- sequestrar sessões
- redirecionar vítimas
- capturar credenciais

Etapas 11 — Analisando Upload de Arquivos

O sistema permitia upload de comprovantes bancários.

Observei a requisição:

```
POST /api/upload
```

Arquivo enviado:

```
comprovante.jpg
```

Realizei um teste simples alterando o nome do arquivo para:

```
shell.php.jpg
```

O servidor aceitou normalmente.

O que isso indicava?

A validação parecia ocorrer apenas:

- pelo nome do arquivo
- não pelo conteúdo real

Impacto potencial

Se explorável:

- execução remota
- webshell
- comprometimento do servidor

Etapa 12 — Investigando Cookies

Ainda na aba Application analisei os cookies.

Encontrei:

```
sessionId=ABCD1234
```

Sem:

- HttpOnly
- Secure

Risco

Qualquer JavaScript poderia acessar o cookie.

Etapa 13 — Descobrindo APIs Ocultas

No filtro XHR do Network identifiquei endpoints interessantes:

```
/api/debug
```

```
/api/export-users
```

```
/api/admin/reports
```

Pensamento durante a análise

“Se o frontend conhece essas rotas, um atacante também consegue descobri-las.”

Resultado Final da Avaliação

Durante o teste utilizando apenas DevTools identifiquei:

Vulnerabilidade	Descoberta
IDOR	Manipulação de requisições
XSS	Campo de pesquisa
APIs internas expostas	Network
Token JWT exposto	Application
Cookies inseguros	Application
Upload inseguro	Network
Comentários sensíveis	Elements
URLs internas	Sources

Conclusão do Projeto

A avaliação demonstrou que apenas utilizando o navegador foi possível:

- mapear APIs
- descobrir funcionalidades ocultas
- manipular requisições
- acessar dados indevidos
- identificar falhas críticas

O teste reforçou que:

- o frontend revela muitas informações
- APIs mal protegidas representam grande risco
- DevTools é uma das ferramentas mais importantes para Pentest Web

Conhecimentos Praticados Neste Laboratório

Durante o laboratório pratiquei:

- análise de tráfego HTTP
- manipulação de requisições
- enumeração de APIs
- identificação de IDOR
- identificação de XSS
- análise de JavaScript
- investigação de tokens
- inspeção de cookies
- reconhecimento de falhas frontend/backend



Autor: Paulo Cesar
Formado em Segurança da informação
linkedin.com/in/paulo-cesar-security